RAPPORT DE PROJET DE FIN D'ÉTUDES

Résidanat TN

Plateforme Web Intelligente de Gestion du Concours de Résidanat

Organisme d'accueil : CIMS

Table des matières

Introduction générale **1**

1 Présentation du cadre de projet **2**
1.1 Introduction 2
1.2 Présentation de l'organisme d'accueil (CIMS) 2
1.2.1 Présentation générale 2
1.2.2 Son rôle stratégique 2
1.2.3 Ses principales missions et activités 3
1.3 Étude de l'existant 3
1.3.1 Présentation des systèmes existants 3
1.3.2 Analyse comparative 5
1.3.3 Problématique dégagée 6
1.3.4 Solution proposée 6
1.4 Architecture adoptée 6
1.5 Méthodologie adoptée 7
1.6 Environnement matériel et logiciel 8
1.6.1 Environnement matériel 9
1.6.2 Environnement logiciel 9
1.7 Conclusion 13

2 Analyse et spécification des besoins **15**
2.1 Introduction 15
2.2 Cahier des charges 15
2.2.1 Objectifs du projet 15
2.2.2 Périmètre du projet 15
2.2.3 Identification des acteurs 16
2.2.4 Besoins fonctionnels 16
2.2.5 Besoins non fonctionnels 17
2.3 Product Backlog 17
2.4 Planification des sprints 21
2.5 Diagramme de cas d'utilisation global 22
2.6 Conclusion 22

3 Sprint 1 « Sécurisation de la plateforme, inscriptions et gestion du concours » **23**
3.1 Introduction 23
3.2 Sprint Backlog 23

3.2.1 Maquettes d'interface 25
3.3 Modélisation fonctionnelle des besoins 27
3.3.1 Identification des acteurs 27
3.3.2 Diagramme de cas d'utilisation globale du sprint 1 27
3.3.3 Diagramme de classe préliminaire 28
3.3.4 Cas d'utilisation « Gérer concours » 28
3.3.5 Cas d'utilisation « S'inscrire au concours » 31
3.4 Modélisation structurelle des besoins 34
3.4.1 Diagramme de classes final 34
3.4.2 Base de données 35
3.4.3 Diagramme de package du cas « Modifier concours » 36
3.5 Tests d'intégration et Validation des APIs 37
3.6 Revue du sprint 38
3.6.1 Principales interfaces 38
3.7 Mise à jour du product backlog 41
3.8 Replanification des sprints 41
3.9 Rétrospective du sprint 42
3.9.1 Tableau de rétrospective 42
3.9.2 ScrumBoard du premier daily 42
3.9.3 ScrumBoard du dernier daily 43
3.10 Mesure d'avancement 44
3.10.1 Burn-down chart 44
3.11 Conclusion 44

Table des figures

1.1 Page d'accueil de la plateforme tunisienne actuelle 4
1.2 Interface de consultation des résultats 4
1.3 Architecture micro-services 7
1.4 Explication de la méthode Scrum 8

2.1 Diagramme de cas d'utilisation global du système 22

3.1 Maquette d'interface : Créer Concours 26
3.2 Maquette d'interface : S'inscrire au concours 26
3.3 Diagramme de cas d'utilisation du Sprint 1 27
3.4 Diagramme de classe préliminaire du Sprint 1 28
3.5 Diagramme raffiné du cas d'utilisation « Gérer concours » 29
3.6 Diagramme de séquence : Créer concours 31
3.7 Diagramme raffiné du cas d'utilisation « S'inscrire au concours » 32
3.8 Diagramme de séquence : S'inscrire au concours 34
3.9 Diagramme de classes final - Sprint 1 35
3.10 Diagramme de package du cas "Modifier concours" 37
3.11 Test d'intégration pour afficher la liste filtrée des concours 38
3.12 Interface de tableau de bord pour la gestion et le suivi des concours 39
3.13 Interface de suivi d'avancement pour un candidat connecté 40
3.14 ScrumBoard du premier daily 43
3.15 ScrumBoard du dernier daily 43
3.16 Burn-down chart du sprint 1 44

Liste des tableaux

1.1 Environnement matériel utilisé 9
1.2 Environnement logiciel Front-end 10
1.3 Environnement logiciel Back-end et Infrastructure 11
1.4 Environnement logiciel dédié à l'Intelligence Artificielle 12
1.5 Outils de conception et de déploiement 13

2.1 Product Backlog du projet 17
2.2 Planification des sprints et vélocité du projet 21

3.1 Sprint backlog du sprint 1 23
3.2 Description textuelle du cas "Gérer concours" 30
3.3 Description textuelle du cas « S'inscrire au concours » (avec validation IA asynchrone) 33
3.4 Product backlog mis à jour 41
3.5 Replanification des sprints 41
3.6 *Tableau de rétrospective* 42

Introduction générale

Le succès d'une institution publique ou éducative repose sur l'efficacité de ses pratiques, techniques, procédures et règles, mises en œuvre par ses équipes dans toutes ses branches. Cela s'avère particulièrement vrai dans les secteurs sensibles de la santé et de l'enseignement supérieur, où la maîtrise du cycle d'organisation des concours nationaux est essentielle pour garantir l'excellence, la transparence et l'équité.

L'organisation d'un concours mobilise de multiples ressources humaines, infrastructurelles et techniques tout en respectant des délais stricts pour les soumissions, les traitements et les délibérations. Or, les processus d'inscription, d'évaluation et de suivi sont souvent complexes, générant des retards, des lourdeurs administratives et compromettant la qualité des résultats. La gestion optimale des concours est donc cruciale pour optimiser les ressources et satisfaire les parties prenantes. Il devient impératif d'adopter des outils performants pour automatiser et fluidifier ces processus.

C'est dans ce cadre que ce rapport de fin d'études présente « Résidanat TN », une application web intelligente que nous avons conçue et développée en étroite collaboration avec l'administration médicale. Cette plateforme conviviale et sécurisée gère intégralement le concours de résidanat en médecine, en permettant aux candidats de soumettre leurs dossiers, de suivre leur avancement en temps réel, et d'interagir efficacement avec les organisateurs.

« Résidanat TN » répond aux besoins des parties prenantes via des fonctionnalités d'inscription automatisée, de validation sécurisée, de gestion des sessions, de publication des résultats et d'assistance aux candidats, le tout sur une architecture scalable et hautement disponible.

Ce rapport s'organise autour d'un premier chapitre introductif, « Présentation du cadre de projet », qui expose l'organisme d'accueil et le contexte professionnel. Les quatre chapitres suivants détaillent les quatre sprints, leur modélisation et leur mise en œuvre, menant à l'incrément final. La conclusion propose un bilan récapitulatif du projet et des perspectives d'évolution de la solution.

Chapitre 1 : Présentation du cadre de projet

1.1 Introduction

Dans ce premier chapitre introductif, nous présentons d'abord le Centre Informatique du Ministère de la Santé (CIMS), organisme d'accueil au sein duquel ce projet de fin d'études a été réalisé, en mettant en avant ses missions et ses activités. Nous exposerons ensuite le contexte général afin de situer le cadre dans lequel s'inscrit notre projet de développement de la plateforme web intelligente pour la gestion du concours de résidanat.

1.2 Présentation de l'organisme d'accueil (CIMS)

1.2.1 Présentation générale

Le Centre Informatique du Ministère de la Santé (CIMS) est un organisme public tunisien stratégique, directement rattaché au Ministère de la Santé Publique. Il représente le pilier technologique de l'État en matière de santé, ayant pour mission principale la conception, la mise en place, l'exploitation et la gestion des systèmes d'information destinés à l'ensemble des structures de santé à l'échelle nationale. Grâce à son expertise et à son positionnement, le CIMS a pour vocation de fournir des solutions technologiques fiables et adaptées aux besoins nationaux, contribuant ainsi à améliorer l'efficacité, la traçabilité et la transparence dans la gestion des services et événements liés à la santé publique.

1.2.2 Son rôle stratégique

Contrairement à une entreprise purement commerciale, le CIMS opère avec un engagement de service public. Il joue un rôle stratégique et moteur dans la modernisation et la transformation digitale du secteur de la santé en Tunisie. Face aux défis actuels de la numérisation des processus étatiques – souvent complexes et sujets à des lourdeurs administratives, comme en témoignent les anciennes procédures de gestion des concours médicaux – le CIMS se positionne comme l'acteur principal chargé d'accompagner les réformes numériques initiées par le Ministère, en garantissant la sécurité, la performance et l'innovation continue.

1.2.3 Ses principales missions et activités

Le champ d'action du CIMS est vaste et couvre de multiples domaines liés à l'ingénierie logicielle et à l'infrastructure informatique de l'État :

— **Développement et intégration :** La conception et le développement d'applications et de plateformes numériques sur mesure pour appuyer la gestion administrative, hospitalière et universitaire.

— **Gouvernance des données :** La centralisation, l'hébergement et la sécurisation rigoureuse des bases de données sensibles liées à la santé publique.

— **Exploitation et support :** La maintenance, la gestion de l'infrastructure réseau et le suivi technique des systèmes informatiques.

— **Pilotage de la transformation :** L'accompagnement technique et stratégique des réformes e-santé.

1.3 Étude de l'existant

1.3.1 Présentation des systèmes existants

Dans le cadre de la gestion des concours médicaux, plusieurs pays ont mis en place des plateformes numériques visant à centraliser les informations et à simplifier les procédures administratives. Dans cette étude, deux références principales ont été analysées : la plateforme officielle tunisienne dédiée au concours de résidanat et la plateforme française du Centre National de Gestion (CNG).

Plateforme tunisienne du concours de résidanat

La plateforme tunisienne constitue le portail officiel d'information relatif au concours de résidanat en médecine. Elle permet aux candidats d'accéder aux textes réglementaires, aux informations concernant les spécialités, aux collèges ainsi qu'aux modalités d'organisation du concours.

La consultation des résultats s'effectue via un formulaire d'identification nécessitant la saisie du numéro de convocation et du numéro de CIN. Une fois ces informations validées, le système affiche les notes obtenues, la moyenne générale ainsi que le rang du candidat. Cependant, la plateforme reste principalement informative et ne propose pas un espace candidat structuré permettant :

— La création d'un compte personnel sécurisé.

— Le dépôt numérique des pièces justificatives.

— Le suivi en temps réel de l'état du dossier.

Par ailleurs, avant la mise en place de la solution proposée dans ce projet, les convocations étaient généralement retirées directement auprès des facultés concernées. Il n'existait pas de système centralisé permettant la génération et le téléchargement d'une convocation numérique sécurisée depuis une plateforme en ligne. Ce fonctionnement implique :

— Des déplacements physiques pour les candidats.

— Une gestion administrative majoritairement manuelle.

— Une absence de suivi numérique personnalisé.

FIGURE 1.1. *Page d'accueil de la plateforme tunisienne actuelle*

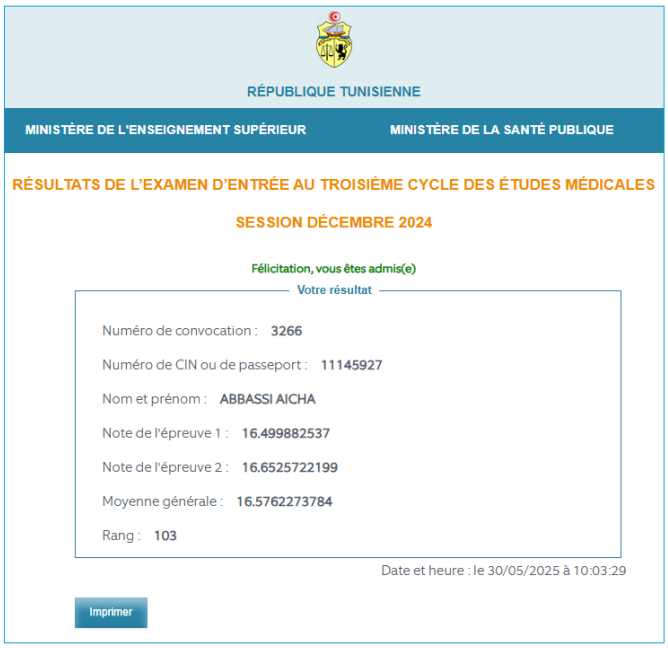

FIGURE 1.2. *Interface de consultation des résultats*

Plateforme française du CNG

La plateforme française du Centre National de Gestion (CNG) propose un système numérique plus structuré pour la gestion des concours médicaux. Elle permet notamment :

— La création d'un espace personnel sécurisé.

— Le dépôt en ligne des pièces justificatives.

— La consultation des convocations.

— La publication des résultats.

Toutefois, l'analyse fonctionnelle met en évidence certaines limites organisationnelles. En particulier :

— L'absence d'un système de notification automatique informant le candidat lorsque son dossier est validé ou lorsque sa convocation est disponible.

— L'absence d'un module structuré de gestion des réclamations permettant au candidat de signaler un problème et d'en suivre le traitement en temps réel.

Ainsi, bien que la plateforme assure la dématérialisation de plusieurs démarches administratives, le suivi dynamique du processus et la communication proactive avec le candidat restent limités.

1.3.2 Analyse comparative

L'étude comparative des deux systèmes permet d'identifier les constats suivants :

Points positifs observés :

— Centralisation des informations officielles.

— Dématérialisation partielle des démarches.

— Publication structurée des résultats.

— Accès sécurisé aux données personnelles.

Limites identifiées :

— Absence d'un système complet de gestion numérique des dossiers dans le contexte tunisien.

— Dépendance aux procédures administratives physiques (retrait des convocations).

— Manque de suivi interactif de l'état d'avancement du dossier.

— Absence d'un mécanisme intégré de gestion des réclamations avec traçabilité.

— Communication limitée entre l'administration et les candidats.

1.3.3 Problématique dégagée

L'analyse de l'existant met en évidence l'absence d'une plateforme intégrée couvrant l'ensemble du cycle de gestion du concours de résidanat, incluant :

— Une inscription entièrement dématérialisée.

— Un suivi structuré et personnalisé du dossier.

— Une génération automatique et sécurisée des convocations.

— Un système formalisé de gestion et de suivi des réclamations.

— Une communication proactive avec les candidats.

Ces constats révèlent un besoin réel de modernisation et de centralisation numérique, afin d'améliorer la transparence, l'efficacité administrative et l'expérience utilisateur.

1.3.4 Solution proposée

Afin de répondre aux limites identifiées dans l'étude de l'existant, la solution proposée consiste en le développement d'une application web centralisée dédiée à la gestion du concours de résidanat en médecine.

Cette plateforme permettra de dématérialiser l'ensemble du processus du concours, depuis l'inscription des candidats jusqu'à la publication des résultats, en passant par la gestion des convocations et des réclamations. Elle offrira aux candidats un espace personnel sécurisé leur permettant d'effectuer leurs démarches administratives en ligne et de suivre l'évolution de leur dossier.

Du côté administratif, la solution proposera un outil de gestion structuré facilitant le traitement des candidatures, la publication des résultats et le suivi global du concours. Un système de notifications permettra d'améliorer la communication entre l'administration et les candidats.

Ainsi, cette solution vise à moderniser le système actuel, à renforcer la transparence du processus et à améliorer l'efficacité administrative, tout en garantissant la sécurité et la fiabilité des données.

1.4 Architecture adoptée

Plusieurs architectures ont été étudiées : monolithique, SOA et microservices. Après analyse, l'architecture microservices a été retenue pour garantir la modularité, la scalabilité, la séparation des responsabilités et la facilité de maintenance.

Les principaux services identifiés sont : Authentification, Gestion des candidatures, IA de validation documentaire, Gestion du concours, Génération des convocations, Importation/Gestion des résultats, Gestion/Classification des réclamations par l'IA, Notifications et Module GPS 3D.

Chaque micro-service dispose de sa propre base de données et communique avec les autres via une API Gateway sécurisée. Cette architecture permet une meilleure évolutivité, une indépendance des modules d'intelligence artificielle et une intégration simplifiée avec d'autres systèmes institutionnels à l'avenir.

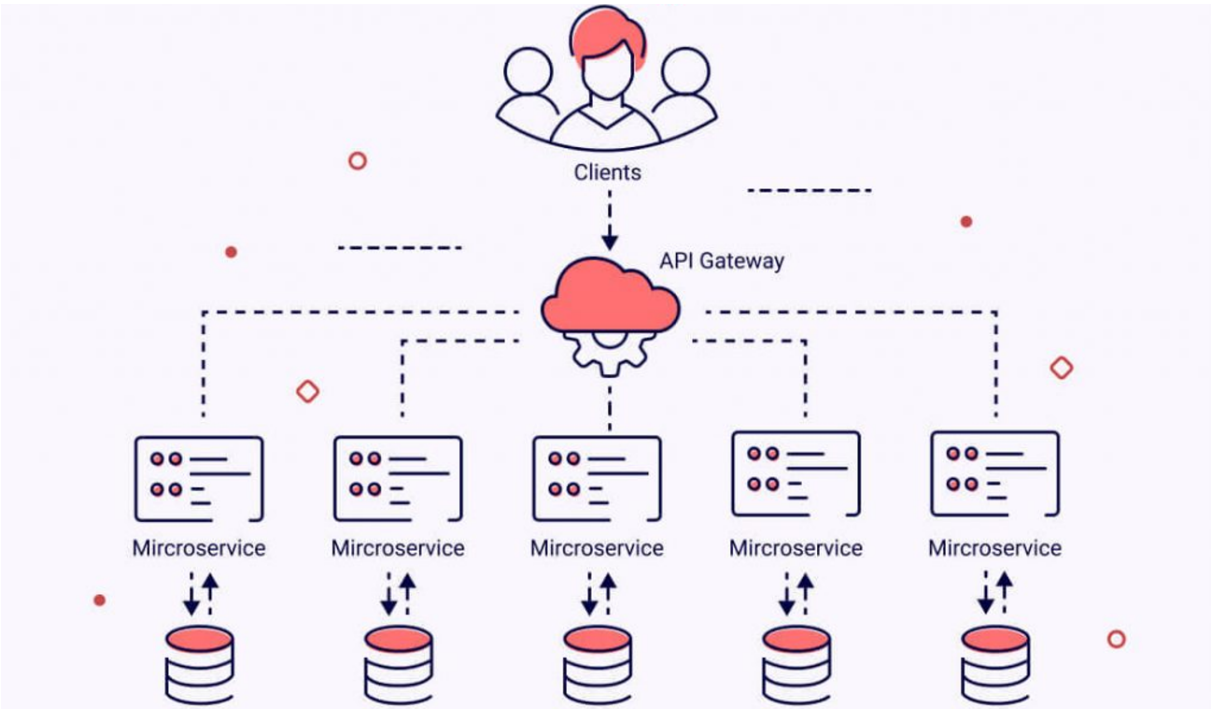

FIGURE 1.3. *Architecture micro-services*

1.5 Méthodologie adoptée

La méthodologie de gestion de projet joue un rôle essentiel dans la réussite du développement d'une application informatique. Dans le cadre de ce projet de développement d'une plateforme web dédiée à la gestion du concours de résidanat en médecine, nous avons choisi d'adopter le framework Scrum.

Scrum est une méthode agile particulièrement adaptée aux projets numériques complexes nécessitant une grande flexibilité, une communication continue entre les membres de l'équipe et une adaptation progressive aux besoins du projet. Cette approche permet de développer l'application de manière incrémentale en livrant progressivement les differentes fonctionnalités.

Le développement du projet est structuré en itérations appelées *sprints*. Chaque sprint correspond à une période de travail durant laquelle un ensemble de fonctionnalités est conçu, développé et testé.

Les sprints prévus pour ce projet sont les suivants :

— **Sprint 1 :** Mise en place du système d'authentification et gestion des utilisateurs.

— **Sprint 2 :** Développement du module de gestion des dossiers candidats.

— **Sprint 3 :** Intégration du module d'intelligence artificielle pour la validation automatique des documents.

— **Sprint 4 :** Gestion des concours et publication des résultats.

— **Sprint 5 :** Intégration de l'assistant 3D et du module intelligent de gestion des réclamations.

— **Sprint 6 :** Développement du module GPS 3D pour la localisation des centres d'examen.

— **Sprint 7 :** Phase de tests, d'optimisation et de sécurisation de l'application.

Des réunions régulières telles que les *Daily Scrum*, les réunions de planification de sprint (*Sprint Planning*), les revues de sprint (*Sprint Review*) et les rétrospectives (*Sprint Retrospective*) permettent d'assurer le suivi de l'avancement du projet, d'identifier les obstacles éventuels et d'ajuster les priorités.

Cette méthodologie favorise une amélioration continue du produit, une meilleure organisation du travail et une adaptation progressive aux besoins spécifiques liés à la gestion du concours de résidanat.

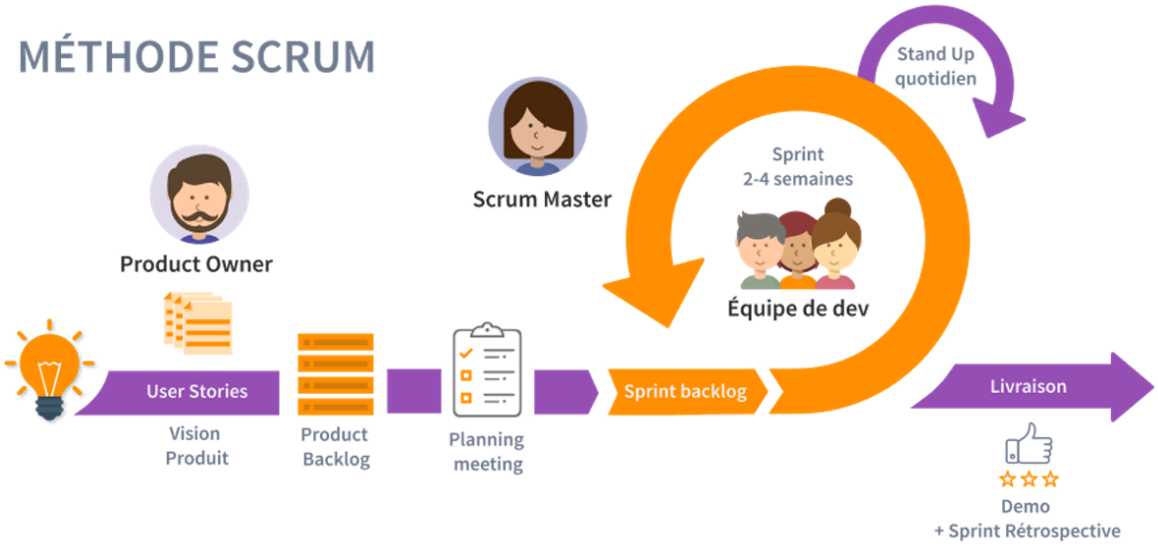

FIGURE 1.4. *Explication de la méthode Scrum*

1.6 Environnement matériel et logiciel

Dans cette section, nous présentons les ressources matérielles et logicielles mobilisées pour le développement de la plateforme Résidanat TN. L'implémentation de notre solution repose sur une architecture en microservices. Nous avons sélectionné un ensemble de frameworks, langages et outils performants pour garantir la modularité, l'évolutivité et l'intégration de l'Intelligence Artificielle.

1.6.1 Environnement matériel

Pour la réalisation de notre projet, nous avons utilisé les postes de travail avec les caractéristiques suivantes :

Nom et prénom	Processeur	Mémoire vive	Disque dur	Système d'exploitation
Syrine Boulabiar	AMD Ryzen 5 3550H with Radeon Vega Mobile Gfx 2.10 GHz	24 GO	512 GO	Windows 11 Professionnel
Sirine Haboubi	Intel(R) Core(TM) i5-1135G7 2.40GHz	20 GO	1 TO	Windows 11 Professionnel

TABLE 1.1. *Environnement matériel utilisé*

1.6.2 Environnement logiciel

• Développement Front-end (Côté Client)

Le tableau ci-dessous présente les technologies et bibliothèques utilisées pour développer l'interface utilisateur interactive de la plateforme.

Nom	Description	Pourquoi l'avoir choisi ?
Angular	Framework frontend développé par Google, basé sur une architecture orientée composants.	Permet de créer une application web monopage (SPA) fluide, modulaire et très performante pour la gestion des formulaires d'inscription.
TypeScript	Sur-ensemble de JavaScript ajoutant le typage statique.	Offre une meilleure sécurité du code, facilite la détection d'erreurs avant la compilation et améliore l'autocomplétion.
Sass / SCSS	Préprocesseur CSS permettant d'ajouter des fonctionnalités avancées au langage CSS.	Facilite la gestion des feuilles de style via l'utilisation de variables, de boucles et d'une structure imbriquée.
SweetAlert2	Bibliothèque JavaScript pour la création de boîtes de dialogue personnalisables.	Utilisée pour concevoir des pop-ups ergonomiques et modernes (notamment pour l'affichage de l'attente du traitement IA).

TABLE 1.2. *Environnement logiciel Front-end*

• Développement Back-end et Architecture Microservices

Cette couche gère la logique métier, la sécurité, le routage et la communication asynchrone entre nos différents services.

Nom	Description	Pourquoi l'avoir choisi ?
Spring Boot	Framework Java simplifiant le développement d'applications autonomes.	Facilite la création rapide de nos microservices (Auth, Dossier, Concours) et la création d'API REST robustes.
Spring Cloud	Suite d'outils pour les architectures distribuées.	Indispensable pour gérer notre API Gateway et notre registre de services (Eureka Discovery).
RabbitMQ	Agent de transfert de messages (Message Broker) open source.	Permet la communication asynchrone entre le service de dossiers et le service IA, évitant ainsi de bloquer l'utilisateur.
PostgreSQL	Système de gestion de base de données relationnelle objet.	Offre une excellente stabilité et une gestion fiable des transactions pour les données des candidats et des concours.

TABLE 1.3. *Environnement logiciel Back-end et Infrastructure*

• Intelligence Artificielle et Traitement d'images

Pour automatiser la vérification des dossiers de candidature, nous avons intégré un microservice dédié à l'IA.

Nom	Description	Pourquoi l'avoir choisi ?
Python	Langage de programmation interprété de haut niveau.	C'est le langage de référence pour l'IA, possédant un vaste écosystème de bibliothèques de traitement d'image et de calcul.
Tesseract OCR	Moteur de reconnaissance optique de caractères développé par Google.	Couplé à OpenCV (pour le prétraitement de l'image), il permet d'extraire efficacement le texte arabe et français des diplômes et des cartes d'identité.

TABLE 1.4. *Environnement logiciel dédié à l'Intelligence Artificielle*

• Outils de conception, développement et déploiement

Les outils suivants ont été indispensables pour coordonner le travail, concevoir les modèles et conteneuriser l'application.

Nom	Description	Pourquoi l'avoir choisi ?
IntelliJ IDEA & VS Code	Environnements de développement (IDE) et éditeurs de code.	IntelliJ est ultra-performant pour l'écosystème Java/Spring, tandis que VS Code est idéal pour Angular et Python.
Docker	Plateforme de conteneurisation logicielle.	Garantit que chaque microservice fonctionne de manière isolée et identique, peu importe l'environnement de déploiement.
Swagger (OpenAPI)	Outil de documentation interactive des API REST.	Simplifie considérablement les tests côté backend et l'intégration des points de terminaison dans le frontend.
GitHub	Plateforme d'hébergement et de gestion de versions.	Indispensable pour centraliser le code source, versionner le travail et collaborer efficacement en équipe.
Lucidchart	Outil de modélisation et de création de diagrammes en ligne.	Choisi pour concevoir et collaborer facilement sur tous les diagrammes UML du projet.
Figma	Outil de conception d'interfaces et de prototypage (UI/UX).	A servi à élaborer les maquettes graphiques de l'application web pour garantir une bonne expérience utilisateur.

TABLE 1.5. *Outils de conception et de déploiement*

1.7 Conclusion

Ce premier chapitre a posé les jalons du projet « Résidanat TN » en présentant l'organisme d'accueil, l'étude de l'existant, ainsi que l'architecture et l'environnement technologique retenus. Nous avons également spécifié le cadre méthodologique qui guidera notre développement.

Le chapitre suivant détaille l'analyse des besoins et la planification détaillée du projet.

Chapitre 2 : Analyse et spécification des besoins

2.1 Introduction

Après avoir présenté le cadre général du projet et l'organisme d'accueil, ce deuxième chapitre se concentre sur l'analyse détaillée des besoins. Nous y définissons les objectifs et le périmètre du projet, identifions les acteurs impliqués et spécifions les fonctionnalités attendues ainsi que les contraintes techniques. Cette étape est cruciale pour structurer le développement et garantir que la solution finale réponde précisément aux attentes des utilisateurs.

2.2 Cahier des charges

2.2.1 Objectifs du projet

Ce projet consiste à concevoir et développer une application web intelligente pour la gestion numérique du concours de résidanat en médecine. La plateforme digitalise l'ensemble du processus, de l'inscription à la publication des résultats, via des mécanismes d'automatisation avancés et des technologies immersives.

Elle permet notamment :

— L'inscription et la gestion sécurisée des candidatures.

— L'validation administrative assistée par l'Intelligence Artificielle.

— La génération automatique de convocations sécurisées.

— L'importation et la publication des résultats.

— La gestion intelligente des réclamations.

— L'assistance à la localisation des centres d'examen via un module GPS 3D.

Les objectifs principaux sont d'améliorer la transparence du processus, l'efficacité administrative, la fiabilité des vérifications, l'expérience utilisateur et d'assurer la modernisation des procédures institutionnelles.

2.2.2 Périmètre du projet

Afin de bien délimiter notre champ d'action, nous avons défini les éléments inclus et exclus de notre système.

Éléments inclus (Dans le périmètre) :

— Gestion des candidatures et analyse IA des documents.

— Gestion des résultats et génération PDF sécurisée avec QR Code.

— Assistant 3D pour la gestion des réclamations.

— Module GPS 3D pour l'orientation vers les centres d'examen.

— Statistiques administratives.

Éléments exclus (Hors périmètre) :

— L'affectation finale des candidats aux différentes spécialités médicales.

— La gestion pédagogique post-admission des résidents.

— L'intégration directe avec d'autres systèmes nationaux externes.

2.2.3 Identification des acteurs

Le système interagit avec trois acteurs principaux :

— **Le Candidat :** Médecin postulant au concours de résidanat.

— **L'Administrateur :** Agent du Ministère de la Santé ou de la faculté, chargé de la gestion du concours.

— **Le Système d'IA :** Module autonome pour l'analyse documentaire, la détection d'incohérences et la priorisation des réclamations.

2.2.4 Besoins fonctionnels

Fonctionnalités Candidat :

— Création de compte sécurisé et authentification.

— Soumission de candidature et téléversement de pièces justificatives.

— Analyse automatique IA (OCR, vérification cohérence, score de conformité).

— Modification du dossier avant validation.

— Suivi en temps réel et notifications.

— Téléchargement de convocation PDF (infos personnelles, centre/salle, QR Code).

— Consultation et impression de résultats.

— Dépôt et suivi de réclamations via assistant 3D.

Fonctionnalités Administrateur :

— Gestion des sessions de concours.

— Importation des résultats Excel avec vérification.

— Publication/dépublication des résultats.

— Validation/rejet des dossiers et gestion des alertes IA.
— Génération des convocations et envoi d'emails automatiques.
— Traitement des réclamations.
— Consultation de statistiques et export de données.

2.2.5 Besoins non fonctionnels

— **Ergonomie :** Interface moderne, intuitive, responsive et institutionnelle.
— **Performance :** Support de pics de connexions, optimisation 3D/IA, gestion de fichiers volumineux.
— **Sécurité :** Chiffrement, HTTPS, rôles/permissions, protection SQL, RGPD.
— **Disponibilité :** Haute disponibilité en périodes critiques.
— **Fiabilité :** Sauvegardes, logs, reprise après incident.

2.3 Product Backlog

Le tableau suivant représente le Product Backlog pour le projet de gestion du concours de résidanat. Il liste les fonctionnalités nécessaires pour atteindre les objectifs du projet, ainsi que leurs ordres de priorité selon les besoins métier.

TABLE 2.1. *Product Backlog du projet Résidanat TN*

N°	Ticket	Histoire	Priorité	Effort	Sprint
1	Pages Publiques	En tant que visiteur, je veux consulter l'accueil, les actualités et les spécialités afin de m'informer sur le résidanat.	Moyenne	5	
2	Création de compte	En tant que candidat, je veux créer un compte via un formulaire sécurisé afin de pouvoir accéder à la plateforme.	Haute	5	
3	Authentification	En tant que candidat/administrateur, je veux m'authentifier afin de pouvoir accéder à mon espace sécurisé.	Haute	8	
4	Déconnexion	En tant qu'utilisateur, je veux me déconnecter afin de sécuriser ma session après utilisation.	Haute	2	

5	Modifier Profil	En tant qu'utilisateur, je veux modifier mon profil afin de mettre à jour mes informations.	Basse	3	
5	Créer Concours	En tant qu'administrateur, je veux créer une nouvelle session de concours afin de lancer une nouvelle session d'inscription.	Haute	5	
6	Gérer Concours	En tant qu'administrateur, je veux modifier, supprimer, publier ou dépublier un concours afin de contrôler sa visibilité et ses détails.	Haute	5	**1**
7	Inscription au concours	En tant que candidat, je veux m'inscrire au concours en saisissant mes informations personnelles et en téléversant mes justificatifs afin de valider ma candidature.	Haute	13	
8	Extraction OCR	En tant que système IA, je veux extraire le texte des documents déposés afin de préparer la vérification automatique.	Haute	13	
9	Analyser anomalies	En tant que système IA, je veux détecter les incohérences entre les documents et le formulaire afin d'améliorer la fiabilité.	Haute	8	
10	Score conformité	En tant que système IA, je veux générer un score de conformité afin d'évaluer automatiquement le dossier.	Moyenne	5	
11	Consulter inscrits	En tant qu'administrateur, je veux visualiser la liste complète des inscrits afin de gérer les dossiers déposés.	Haute	5	
12	Visualiser alertes	En tant qu'administrateur, je veux consulter les alertes générées par l'IA afin de faciliter la validation des dossiers.	Haute	5	

13	Valider/Rejeter	En tant qu'administrateur, je veux valider ou rejeter un dossier afin d'assurer sa conformité aux exigences.	Haute	5	**2**
14	Notifier candidat	En tant que système, je veux notifier le candidat par email afin qu'il puisse corriger son dossier s'il est incomplet.	Moyenne	5	
15	Modifier Dossier	En tant que candidat, je veux modifier mon dossier avant sa validation afin de corriger d'éventuelles erreurs.	Haute	5	
16	État du dossier	En tant que candidat, je veux consulter l'état de mon dossier afin de suivre l'avancement de ma candidature.	Haute	3	
17	Convocation PDF	En tant que système, je veux générer une convocation PDF sécurisée avec QR Code afin de permettre l'accès à l'examen.	Haute	8	**3**
18	Télécharger convocation	En tant que candidat, je veux télécharger ma fiche de convocation afin de me présenter au centre d'examen.	Haute	3	
19	Importer résultats	En tant qu'administrateur, je veux importer un fichier Excel de notes afin d'intégrer massivement les résultats du concours.	Haute	13	**4**
20	Vérifier données	En tant que système, je veux vérifier la cohérence des données importées afin d'éviter les erreurs ou doublons de notes.	Haute	8	
21	Publier résultats	En tant qu'administrateur, je souhaite publier les notes officielles afin d'informer les candidats de leurs résultats.	Haute	5	
22	Consulter résultat	En tant que candidat, je veux consulter mon statut (admis/rejeté) et mon classement afin de connaître ma réussite.	Haute	5	

23	Imprimer fiche	En tant que candidat, je veux imprimer ma fiche détaillée officielle afin d'avoir une preuve de mon résultat.	Moyenne	5	
24	Emails automatiques	En tant que système, je veux envoyer des notifications automatiques afin d'informer les admis des prochaines étapes.	Haute	5	
25	Accès Réclamations	En tant que candidat, je veux accéder au module de réclamation afin de signaler un problème ou un litige.	Haute	3	
26	Assistant 3D IA	En tant que candidat, je veux interagir avec l'assistant 3D afin d'être guidé dans la saisie de ma réclamation.	Haute	13	**5**
27	Classifier IA	En tant que système IA, je veux classifier automatiquement les réclamations afin d'optimiser leur traitement.	Haute	13	
28	Prioriser IA	En tant que système IA, je veux attribuer un niveau de priorité aux réclamations afin d'organiser le travail de l'admin.	Haute	8	
29	Traiter réclamation	En tant qu'administrateur, je veux consulter et répondre aux réclamations afin de résoudre les problèmes signalés.	Haute	8	
30	Suivre réclamation	En tant que candidat, je veux consulter l'état de ma réclamation afin de suivre l'évolution de son traitement.	Moyenne	3	**6**
31	Centre d'examen 3D	En tant que candidat, je veux visualiser le centre d'examen en 3D afin de le localiser facilement.	Haute	13	
32	Simuler itinéraire	En tant que candidat, je veux simuler un itinéraire vers mon amphithéâtre afin de mieux préparer mon déplacement.	Moyenne	13	

33	Statistiques KPI	En tant qu'administrateur, je veux consulter les statistiques globales afin d'analyser l'état du concours.	Haute	8	**7**
34	Exporter données	En tant qu'administrateur, je veux exporter les listes de candidats afin d'effectuer des traitements administratifs.	Moyenne	5	
35	Modifier Profil	En tant qu'utilisateur, je veux modifier mes préférences et mon profil afin de mettre à jour mes informations personnelles.	Basse	3	

2.4 Planification des sprints

Afin d'assurer une gestion efficace du projet sur la période fixée pour notre stage, nous avons opté pour une approche itérative basée sur la méthode Agile Scrum, en divisant le travail en 7 sprints. Cette approche a permis une répartition réfléchie de la vélocité entre les différentes phases du projet, favorisant ainsi une progression optimale.

Le tableau 3.5 illustre la répartition des histoires utilisateurs (User Stories) pour chaque itération ainsi que sa vélocité correspondante (somme des efforts estimés).

TABLE 2.2. *Planification des sprints et vélocité du projet*

N° Sprint	N° User Story	Vélocité
Sprint 1	De 1 à 11	69
Sprint 2	De 12 à 16	23
Sprint 3	De 17 à 18	11
Sprint 4	De 19 à 22	31
Sprint 5	De 23 à 29	55
Sprint 6	De 30 à 32	29
Sprint 7	De 33 à 35	16

2.5 Diagramme de cas d'utilisation global

Afin de synthétiser les interactions entre les différents acteurs identifiés et les grandes fonctionnalités de notre plateforme, nous avons modélisé le diagramme de cas d'utilisation global illustré par la figure 2.1. Ce diagramme englobe l'ensemble du périmètre du projet, de l'inscription au suivi des réclamations, et met en évidence la valeur ajoutée apportée par le système d'Intelligence Artificielle de manière asynchrone.

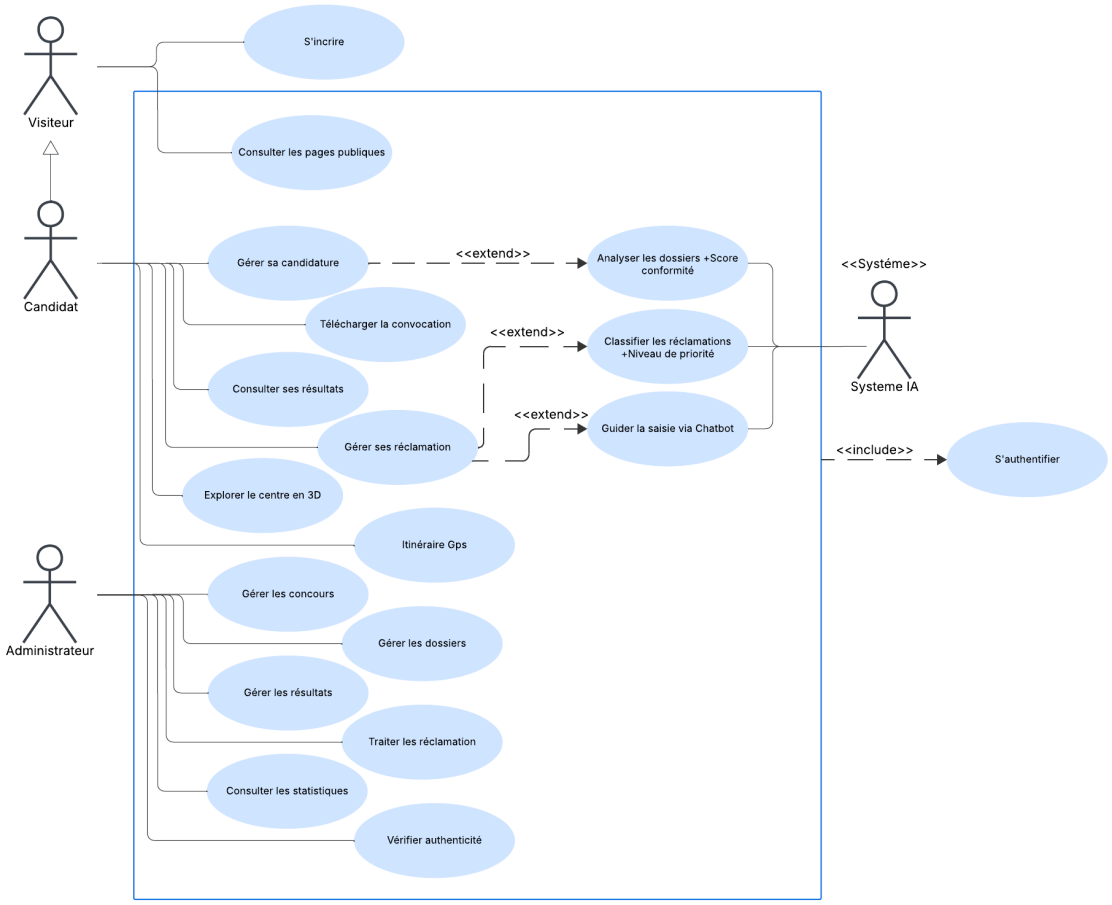

FIGURE 2.1. *Diagramme de cas d'utilisation global du système*

2.6 Conclusion

Ce chapitre a permis de poser les bases analytiques du projet en spécifiant les besoins fonctionnels et non fonctionnels, ainsi qu'en structurant le travail via le Product Backlog et la planification des sprints. Ces éléments constituent la feuille de route pour le développement technique de la solution. Le chapitre suivant détaillera l'implémentation effective du premier sprint.

Chapitre 3 : Sprint 1 « Sécurisation de la plateforme, inscriptions et gestion du concours »

3.1 Introduction

Le premier sprint de notre démarche Scrum constitue une étape cruciale pour la plateforme « Résidanat TN ». Il pose les bases fondamentales du système, notamment à travers la mise en place de l'architecture de sécurité et le développement des fonctionnalités d'inscription. L'objectif principal de ce sprint est de permettre aux candidats de créer un compte sécurisé et à l'administration de configurer les sessions de concours. Ce chapitre explore les phases de conception et de réalisation de ce premier incrément, tout en soulignant l'importance de la validation des mécanismes d'authentification pour garantir la protection des données médicales et personnelles.

3.2 Sprint Backlog

Le tableau ci-dessous présente le Sprint Backlog détaillé pour ce premier sprint. Il décompose chaque récit utilisateur (User Story) en tâches de conception, de développement et de test, tout en précisant l'effort estimé en jours.

TABLE 3.1. *Sprint backlog du sprint 1*

N° Story	Id	User Story	Tâches techniques	Jours
1	1	En tant que visiteur, je veux consulter l'accueil, les actualités et les spécialités afin de m'informer sur le résidanat.	1. Diagramme de cas d'utilisation	3
			2. Maquette d'interface	
			3. Diagramme de classe préliminaire	
			4. Diagramme de classe final	
			5. Implémentation Front-end	
			6. Tests	
			7. Documentation	
De 2 à 4	2	S'authentifier et sécuriser l'accès à l'espace personnel	1. Diagramme de cas d'utilisation	5
			2. Maquette d'interface	

			3. Diagramme de classe préliminaire	
			4. Diagramme de séquence (Authentification)	
			5. Diagramme de séquence de conception (JWT)	
			6. Diagramme de classe raffiné	
			7. Configuration Session & Sécurité Back-end	
			8. Implémentation API Auth et Stockage DB	
			9. Intégration Front-end	
			10. Tests sécurité	
			11. Documentation	
De 5 à 6	3	Gérer les sessions de concours (Admin).	1. Diagramme de cas d'utilisation Admin	5
			2. Maquette d'interface (Espace Admin)	
			3. Diagramme de classe (Session/Concours)	
			4. Diagramme de séquence (CRUDS Concours)	
			5. Implémentation Front-end et Back-end	
			6. Tests	
			7. Documentation	
7	4	En tant que candidat, je veux m'inscrire au concours en téléversant mes justificatifs.	1. Diagramme de cas d'utilisation	4
			2. Maquette d'interface	
			3. Diagramme de classe	
			4. Diagramme de séquence (Dépôt dossier)	
			5. Sécurisation du téléversement (Mime-type, Taille)	

			6. Implémentation Système de fichiers	
			7. Développement API Dossier & Vues Front	
			8. Tests	
			9. Documentation	
De 8 à 10	5	Analyse IA et Score de conformité pour la validation des dossiers.	1. Étude technique et choix des outils IA	6
			2. Diagrammes de classes et Workflow IA	
			3. Diagramme de séquence (Pipeline IA)	
			4. Prétraitement des images (Grayscaling)	
			5. Implémentation du moteur OCR	
			6. Implémentation algo de détection d'anomalies	
			7. Implémentation algo calcul du score	
			8. Intégration avec le module Inscription	
			9. Évaluation de la précision (Metrics IA)	
			10. Documentation technique détaillée	

3.2.1 Maquettes d'interface

La conception des maquettes nous permet d'avoir un aperçu visuel des futures pages de l'application avant d'entamer la phase de développement.

— **Maquette d'interface « Créer Concours » :**

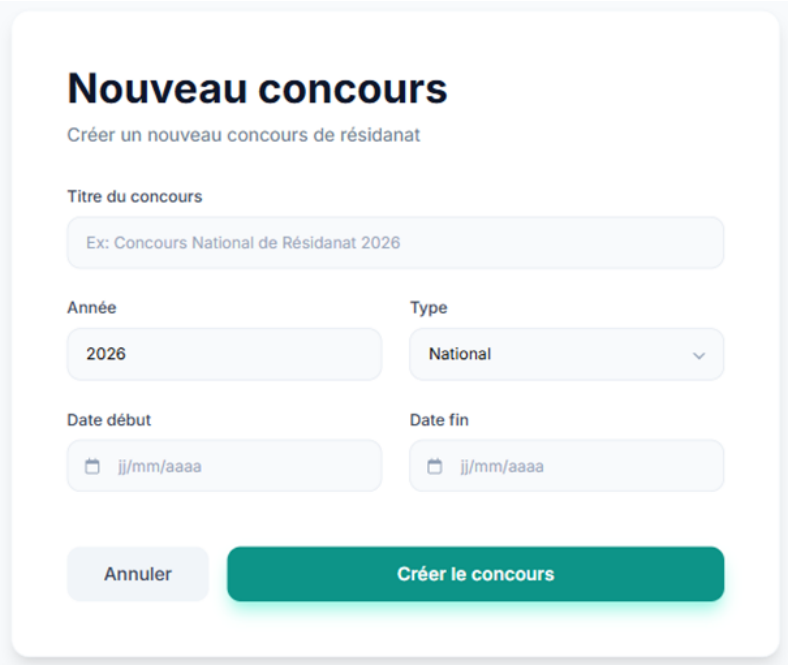

FIGURE 3.1. *Maquette d'interface : Créer Concours*

— **Maquette d'interface « S'inscrire au concours » :**

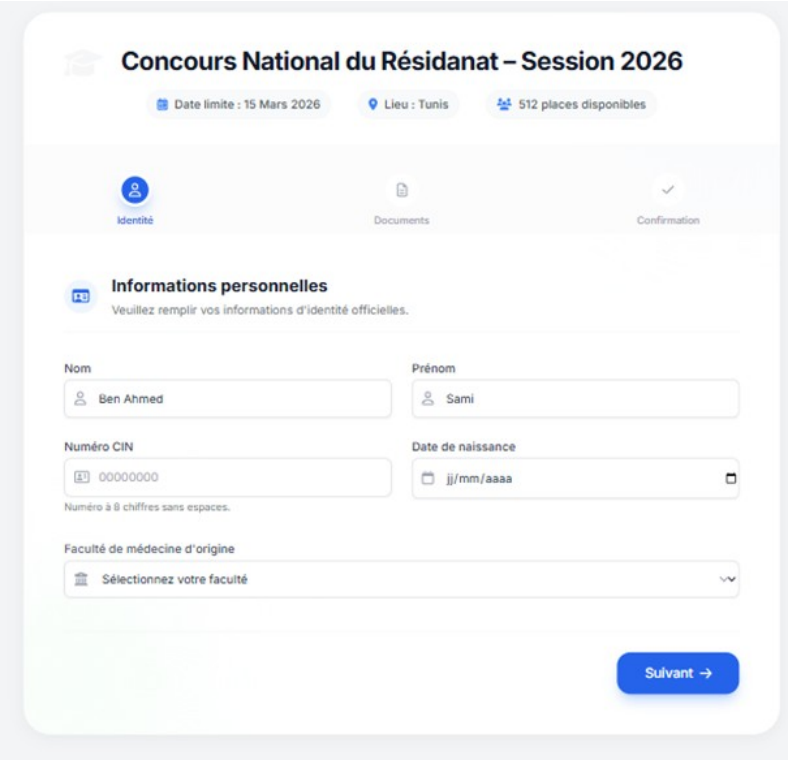

FIGURE 3.2. *Maquette d'interface : S'inscrire au concours*

3.3 Modélisation fonctionnelle des besoins

3.3.1 Identification des acteurs

Les acteurs suivants ont été identifiés pour interagir avec l'application :

— **Le visiteur**

— **Le candidat**

— **L'administrateur**

3.3.2 Diagramme de cas d'utilisation globale du sprint 1

Le diagramme de cas d'utilisation ci-dessous illustre les fonctionnalités fournies aux acteurs de l'application.

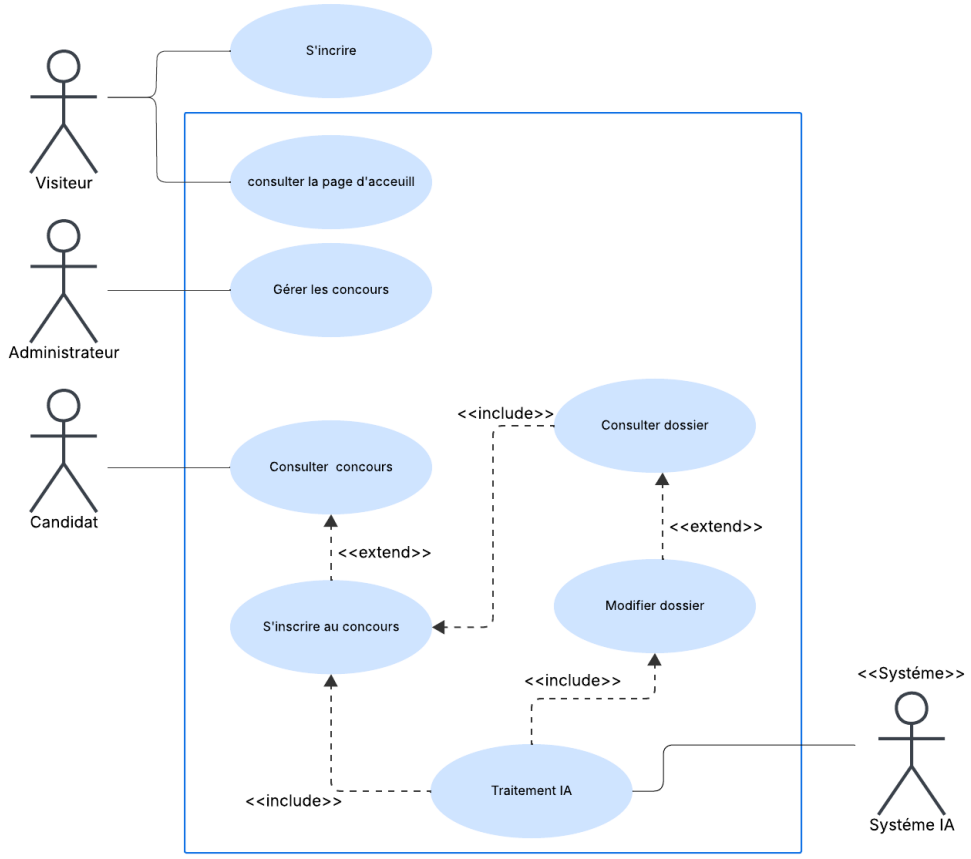

FIGURE 3.3. *Diagramme de cas d'utilisation du Sprint 1*

3.3.3 Diagramme de classe préliminaire

Le diagramme ci-dessous représente le diagramme de classe préliminaire de notre premier sprint.

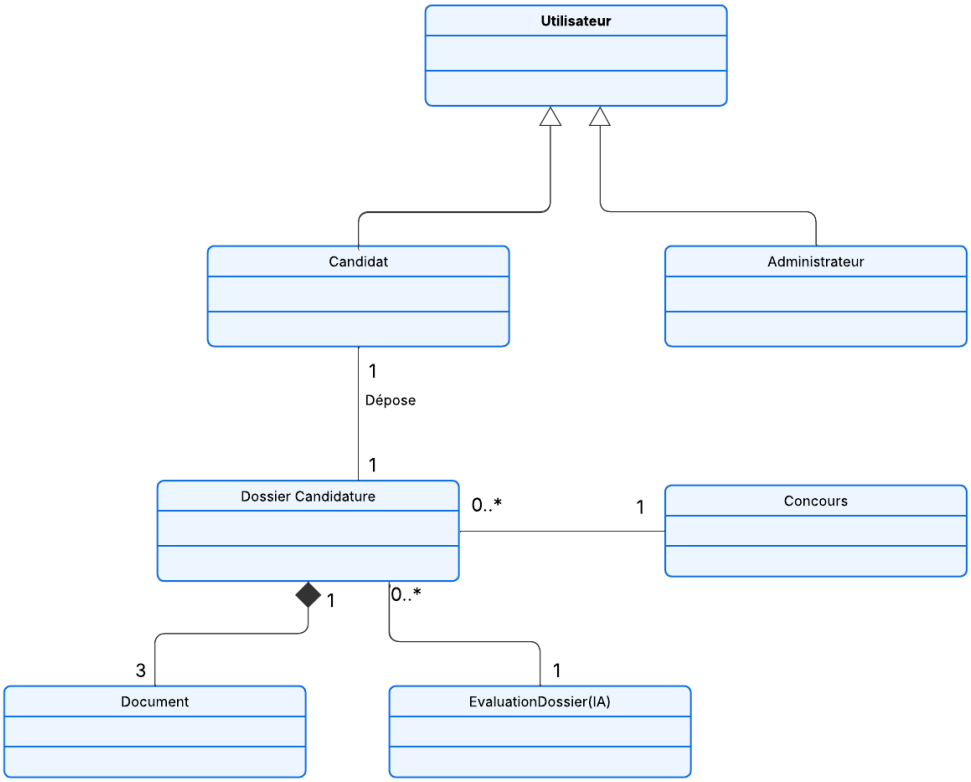

FIGURE 3.4. *Diagramme de classe préliminaire du Sprint 1*

3.3.4 Cas d'utilisation « Gérer concours »

3.3.4.1 Diagramme de cas d'utilisation raffiné

Le diagramme de la figure 3.5 représente de façon plus détaillée le cas d'utilisation « Gérer concours ».

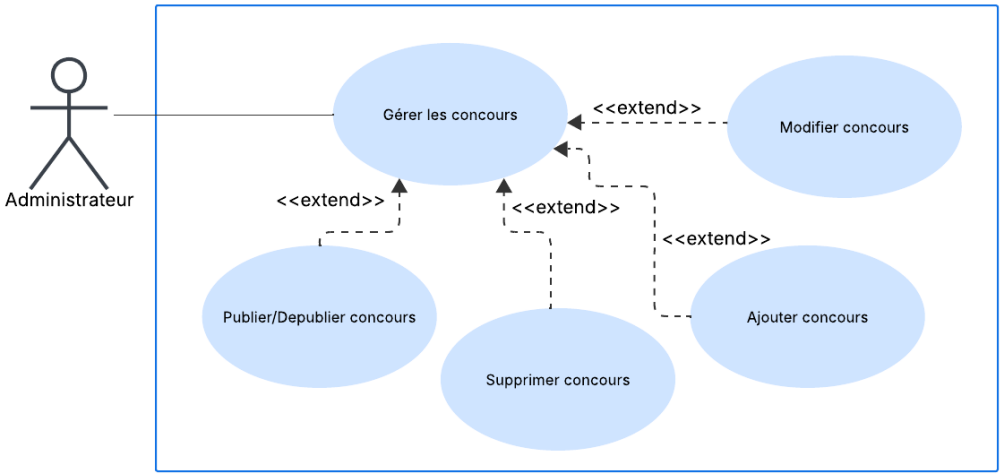

FIGURE 3.5. *Diagramme raffiné du cas d'utilisation « Gérer concours »*

3.3.4.2 Description textuelle

Le tableau 3.2 détaille le processus de gestion de concours en mettant l'accent sur les actions de l'administrateur.

TABLE 3.2. *Description textuelle du cas "Gérer concours"*

Identification	**Nom du cas :** Créer concours **Objectif :** Permettre à l'administrateur d'ajouter une nouvelle session de concours au système. **Acteur :** L'administrateur
Pré-condition	L'administrateur doit être authentifié et connecté à son espace d'administration.
Enchaînement nominal	1) L'administrateur clique sur la rubrique "Gestion des concours". 2)Le système récupère et affiche la liste des concours existants. 3) L'administrateur clique sur le bouton "Nouveau concours". 4) Une interface de saisie s'affiche avec les champs requis (Titre, date de début, date de fin, état). 5) L'administrateur saisit les données du nouveau concours et valide. 6)Le système enregistre le concours, affiche un message de succès et actualise la liste.
Enchaînement alternatif	1) **En (4) :** Si les dates saisies sont incohérentes (ex : date de fin avant la date de début) ou si des champs obligatoires sont vides, le système affiche une erreur et bloque la validation. 2) **En (5) :** En cas de problème de connexion au serveur lors de l'enregistrement, un message d'alerte s'affiche invitant l'administrateur à réessayer.
Post-condition	Les informations du nouveau concours sont sauvegardées en base de données et la liste est mise à jour.

3.3.4.3 Diagramme de séquence

La figure 3.6 illustre la dynamique des échanges pour la création, la modification et la consultation des sessions de concours par l'administrateur au sein du système.

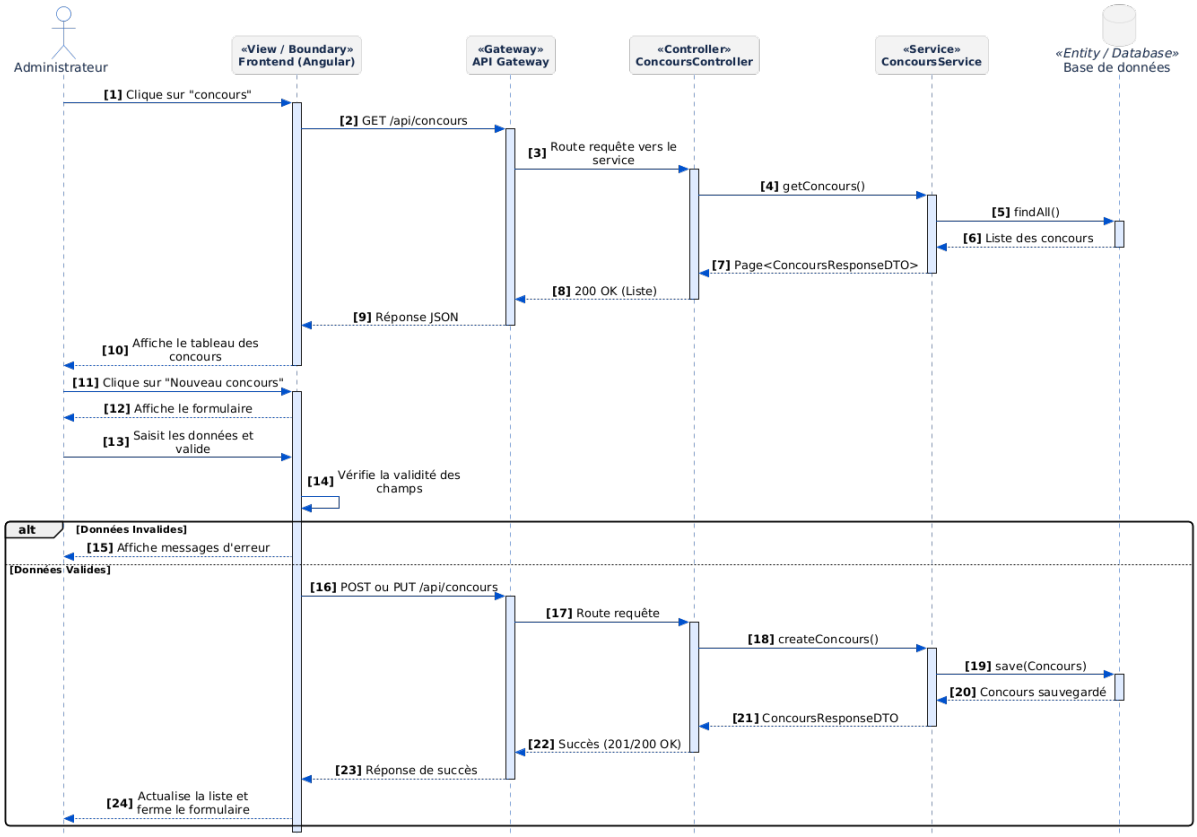

FIGURE 3.6. *Diagramme de séquence : Créer concours*

3.3.5 Cas d'utilisation « S'inscrire au concours »

3.3.5.1 Diagramme de cas d'utilisation raffiné

Le diagramme de la figure 3.7 modélise de manière détaillée le cas d'utilisation « S'inscrire au concours ». Il met en évidence les cas d'utilisation inclus obligatoires (remplir le formulaire, téléverser les justificatifs, s'authentifier) ainsi que les cas d'extensions liés à la gestion des erreurs de saisie ou de fichiers.

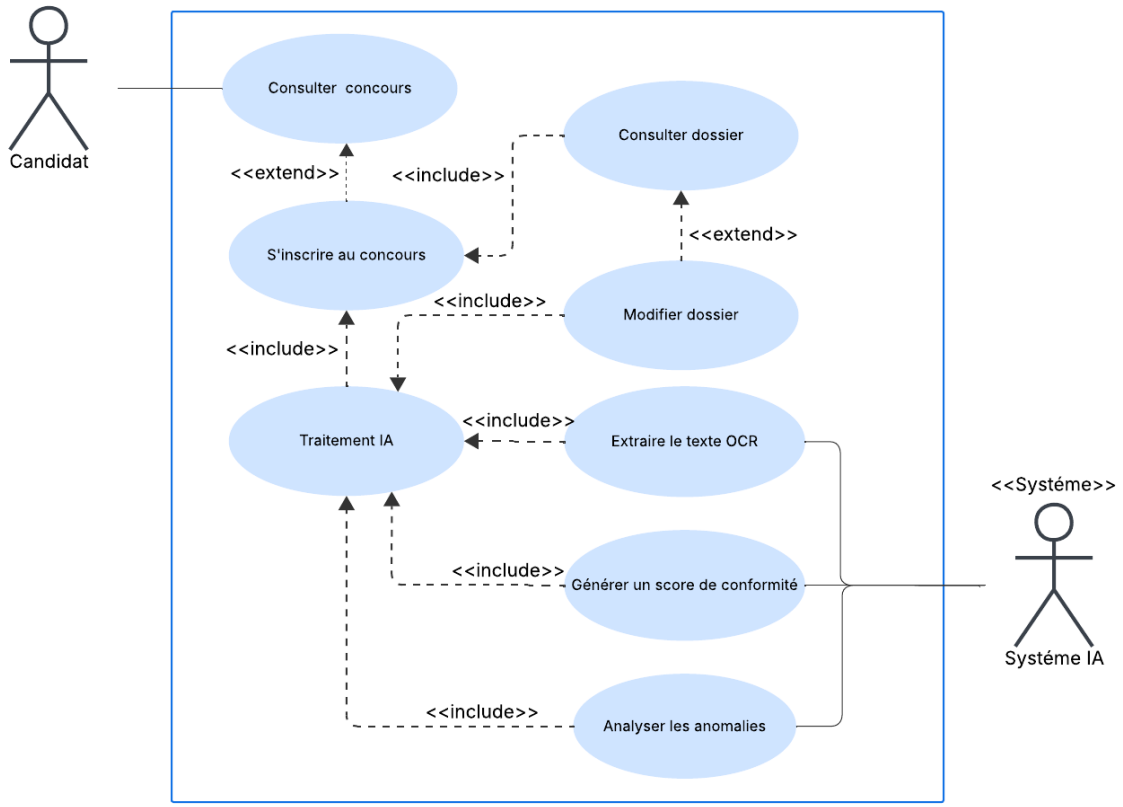

FIGURE 3.7. *Diagramme raffiné du cas d'utilisation « S'inscrire au concours »*

3.3.5.2 Description textuelle

Le tableau 3.3 détaille le processus de candidature en mettant l'accent sur l'intervention asynchrone du microservice d'Intelligence Artificielle pour la vérification des documents.

TABLE 3.3. *Description textuelle du cas « S'inscrire au concours » (avec validation IA asynchrone)*

Identification	**Nom du cas :** S'inscrire au concours **Objectif :** Permettre au candidat de soumettre son dossier tout en déclenchant une analyse OCR asynchrone via des scripts Python pour évaluer la conformité des documents. **Acteur :** Le candidat
Pré-condition	Le candidat doit être authentifié et une session de concours doit être ouverte.
Enchaînement nominal	1) Le candidat accède au formulaire d'inscription et saisit ses données (CIN, Nom, Prénom, Date de naissance, Faculté, etc.). 2) Il téléverse ses documents justificatifs (CIN, Diplôme , Photo d'identité). 3) Le candidat valide sa candidature, ce qui enregistre les fichiers via le système de stockage (*FileStorageService*). 4) Le backend (*dossier-service*) publie une requête contenant l'identifiant du dossier et le chemin des images dans une file d'attente **RabbitMQ** (`q.ia.request`). 5) Le microservice d'IA (`ia-validation-service`) intercepte le message et lance le processus en arrière-plan. 6) Le service exécute dynamiquement le script Python approprié selon le document (`test_diplome_ia.py` pour le diplôme, `test_ia.py` pour la CIN). 7) Le script extrait le texte (OCR) et le compare aux données du candidat. 8) Le service IA renvoie le résultat formaté via RabbitMQ, mettant ainsi à jour le statut du dossier.
Enchaînement alternatif	1) **En (1) et (2) :** Si les données saisies sont invalides, le système bloque l'envoi. 2) **En (7) :** Si le script Python échoue, le service Java intercepte l'erreur et attribue un score de 0.0. 3) **En (8) :** Si des anomalies sont détectées, la variable `anomalies` alerte l'administrateur.
Post-condition	Le dossier est créé, les fichiers sont stockés et l'analyse IA est planifiée.

3.3.5.3 Diagramme de séquence

La figure 3.8 illustre la dynamique des échanges et la vérification des conditions entre le candidat, l'interface et le système backend lors de la soumission d'une candidature, incluant le traitement par l'IA.

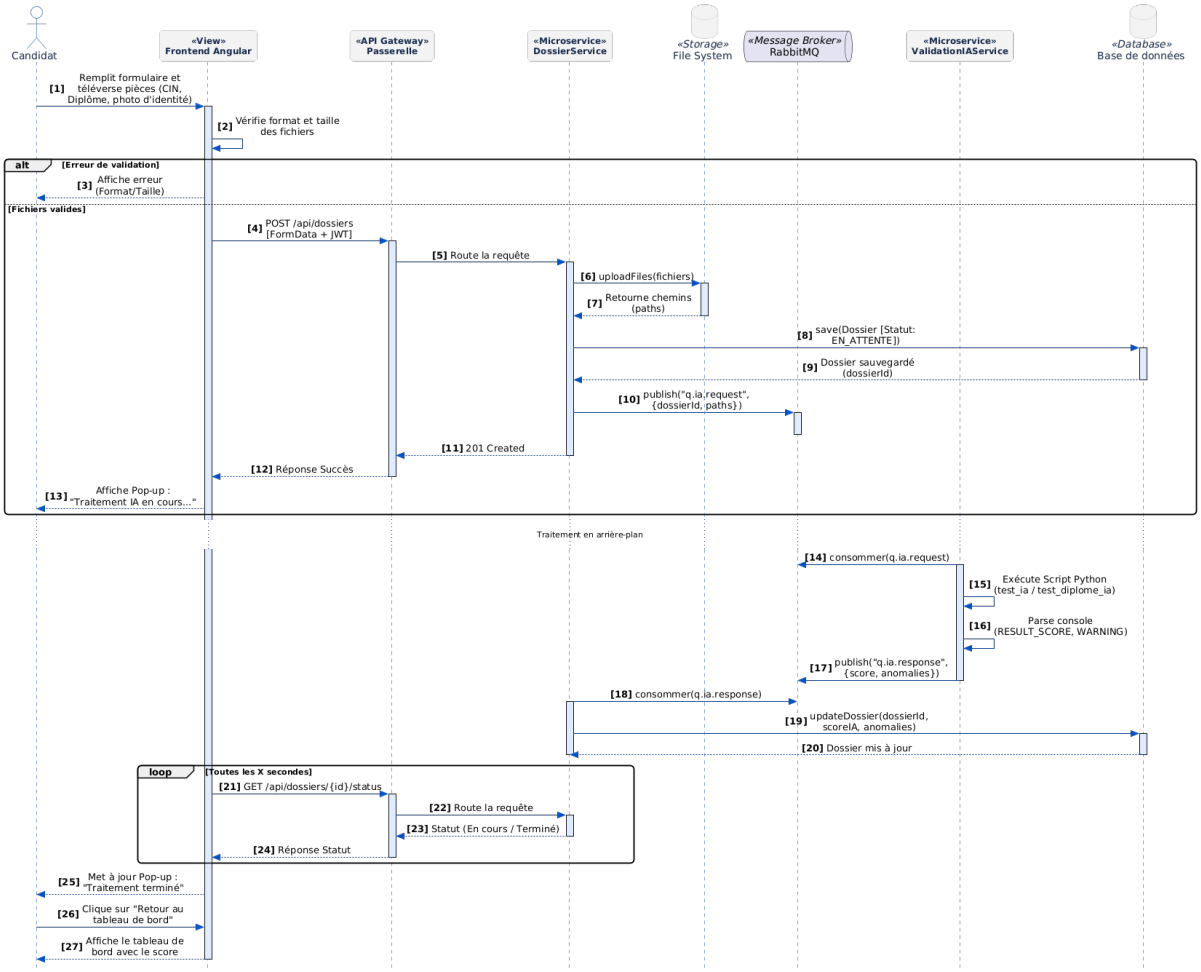

FIGURE 3.8. *Diagramme de séquence : S'inscrire au concours*

3.4 Modélisation structurelle des besoins

3.4.1 Diagramme de classes final

Le diagramme de classe final suivant représente les classes et les relations entre les entités au cours du sprint 1.

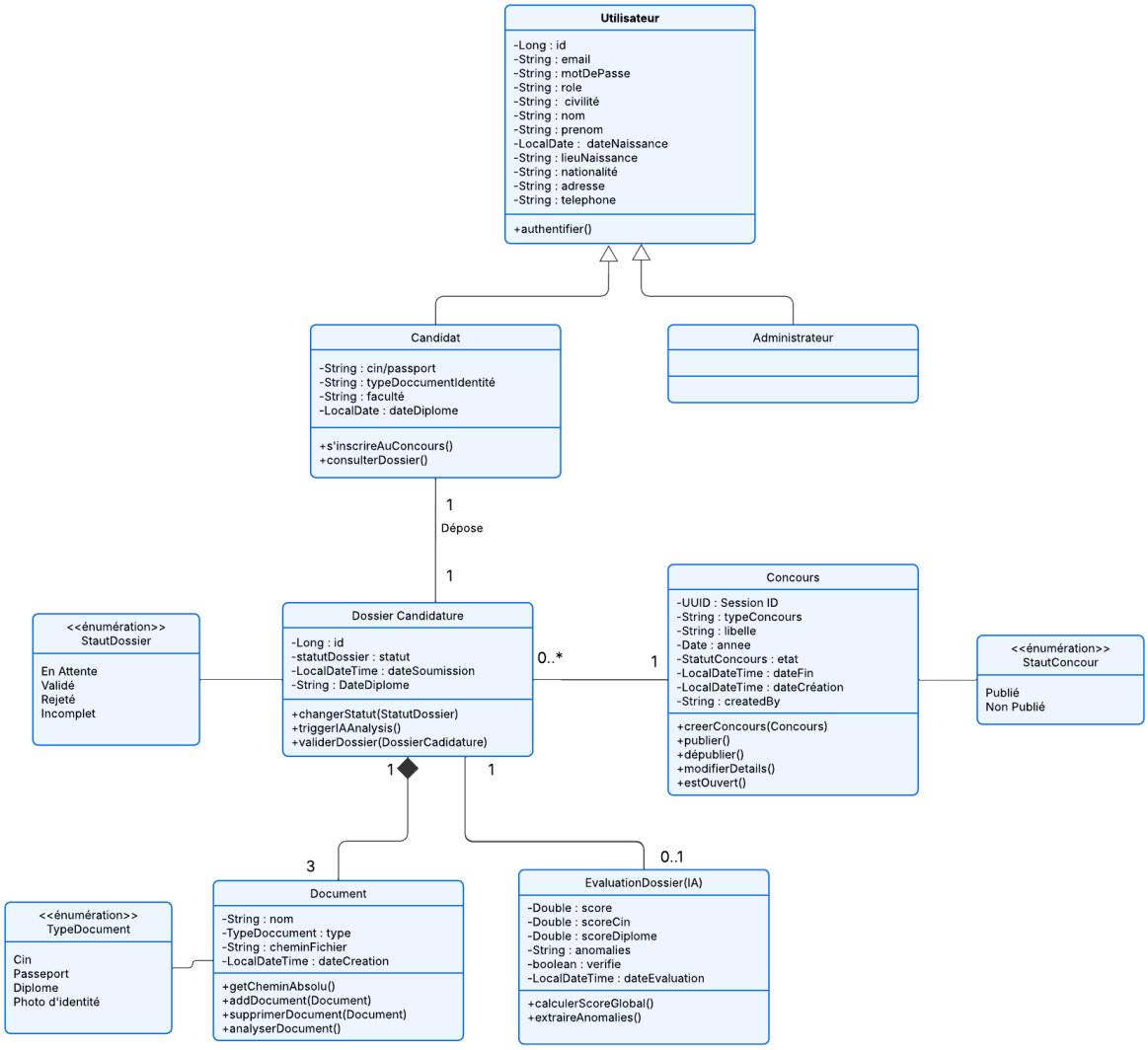

FIGURE 3.9. *Diagramme de classes final - Sprint 1*

3.4.2 Base de données

Au cours de notre premier sprint, nous avons adopté une approche basée sur des micro-services, où chaque service possède sa propre base de données. Dans cette architecture, il n'existe pas de clés étrangères physiques entre les tables appartenant à des micro-services différents. Chaque micro-service gère ses données de manière autonome et indépendante. Les relations entre les entités transverses sont gérées par des identifiants uniques (IDs) qui sont échangés entre les micro-services lors des interactions nécessaires.

Une particularité de cette configuration est l'utilisation d'attributs dits **"transients"** (ou de référence) dans certaines tables. Ces attributs permettent de stocker les identifiants d'entités distantes afin de faciliter le référencement et l'accès aux données gérées par d'autres micro-services. Bien que ces identifiants agissent comme des références logiques, ils ne correspondent pas à des clés étrangères réelles avec contraintes d'intégrité référentielle au niveau de la base de données, préservant ainsi le couplage faible entre les services.

Par exemple, au niveau de la table *« DossierCandidature »*, l'attribut *« candidatId »* fait référence à l'identifiant de l'utilisateur stocké dans la table *« Utilisateur »* du micro-service d'authentification (**auth-service**).

Voici la structure des principales tables de bases de données pour le sprint 1 :

— **Utilisateur**(id, email, motDePasse, role, civilite, nom, prenom, dateNaissance, lieuNaissance, nationalite, adresse, cin, typeDocumentIdentite, telephone, faculte, dateDiplome)

— **Concours**(Session Id, typeConcours, libelle, annee, etat, dateDebut, dateFin, dateCreation, createdBy)

— **DossierCandidature**(id, *candidatId*, *concoursId*, statut, dateSoumission, dateDiplome)

— **Document**(id, nom, type, dateCreation, cheminFichier, *dossier_id*)

— **EvaluationIA**(id, score, scoreCin, scoreDiplome, anomalies, verifie, dateEvaluation, *dossier_id*)

3.4.3 Diagramme de package du cas « Modifier concours »

Le diagramme de package ci-dessus montre l'architecture globale mise en place pour le cas d'utilisation « Modifier concours ». Ce dernier implique la partie front-end et la partie back-end qui est structurée sous forme de micro-services.

La partie front-end présente une boîte de dialogue (composant `modifierDialogModel`) regroupant un formulaire réactif pour la modification des informations d'un concours existant. Après la saisie et l'appui sur le bouton de modification, la méthode `validerFormulaire(concours)` se déclenche et fait appel à la méthode `updateConcours(id, c)` du service Angular `ConcoursService`. Ce service utilise le protocole HTTP pour communiquer avec le `Gateway`. Ce dernier se charge d'acheminer la requête vers le micro-service « `concours-service` » après avoir récupéré son adresse dynamiquement auprès du service de découverte (Discovery Service) « `Eureka` ».

Ensuite, au niveau du back-end, la requête associée à la méthode `updateConcours(id, dto)` de la classe `ConcoursController` s'exécute. Le contrôleur délègue le traitement métier au service `ConcoursService`, qui importe à son tour l'interface `ConcoursRepository`. Cette dernière hérite de `JpaRepository` et permet finalement d'exécuter la transaction pour mettre à jour les données dans la base de données au niveau de l'entité `Concours`.

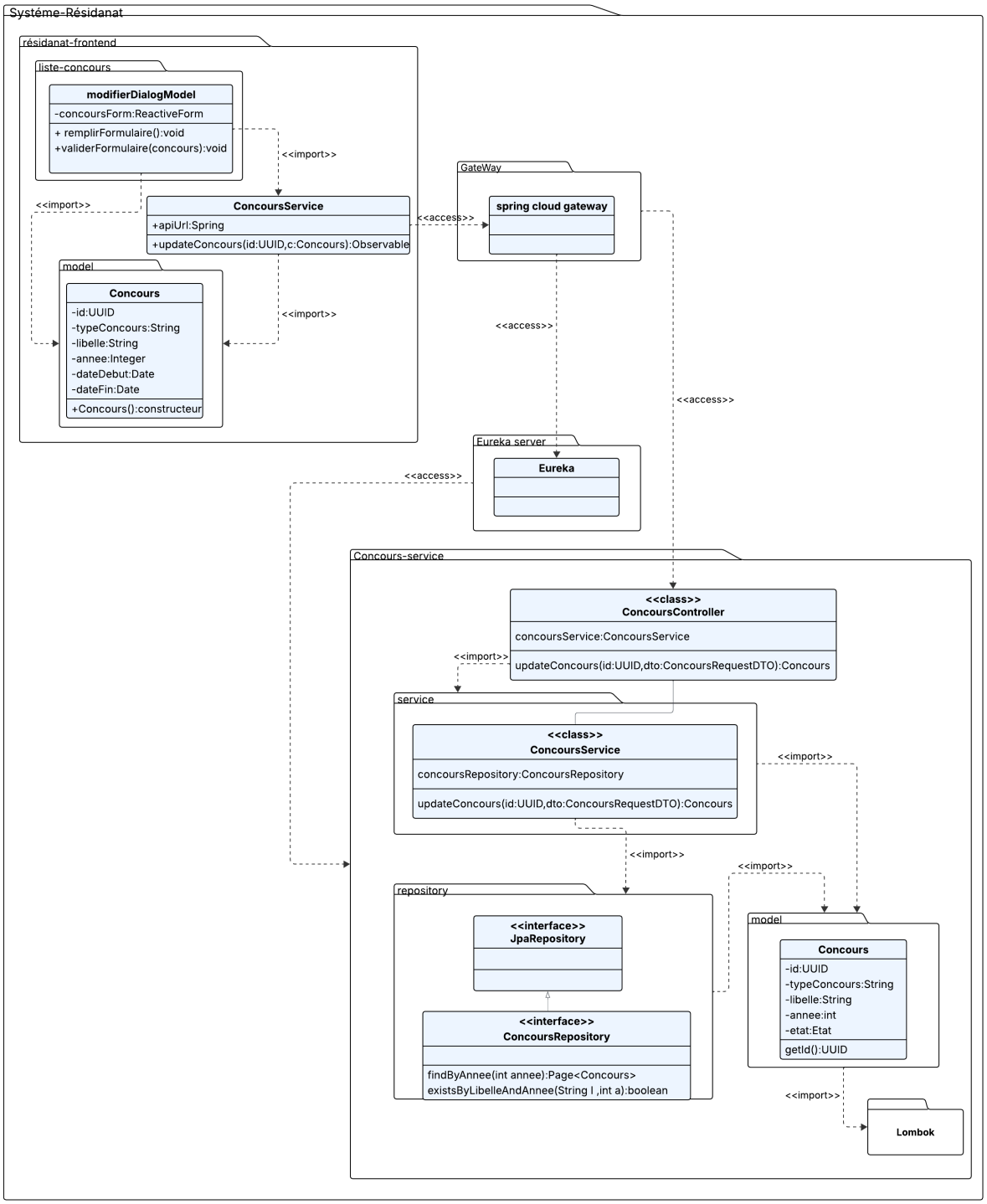

FIGURE 3.10. *Diagramme de package du cas "Modifier concours"*

3.5 Tests d'intégration et Validation des APIs

Les tests des APIs (Endpoints) permettent de vérifier le bon fonctionnement d'une partie précise d'une application web, et de s'assurer de la bonne communication entre le frontend et notre architecture micro-services. Au lieu d'utiliser un outil de test externe classique (comme Postman ou Thunder Client), nous avons opté pour le framework **Swagger (OpenAPI 3.0)** directement intégré au sein de

nos micro-services via Spring Boot. Ce choix stratégique permet de générer automatiquement une documentation vivante et interactive, et de tester le bon fonctionnement de notre code directement depuis le navigateur (en incluant de manière transparente la gestion de la sécurité par Token JWT). Pour illustrer cette démarche, nous avons testé l'endpoint de **« Consultation de la liste des concours avec filtres et pagination »**. Ce test permet de vérifier la capacité de notre backend à intercepter des paramètres de recherche complexes (tels que l'année, le type et l'état de publication) et à renvoyer la liste structurée des concours correspondants depuis la base de données, le tout en exigeant une authentification valide de l'utilisateur.

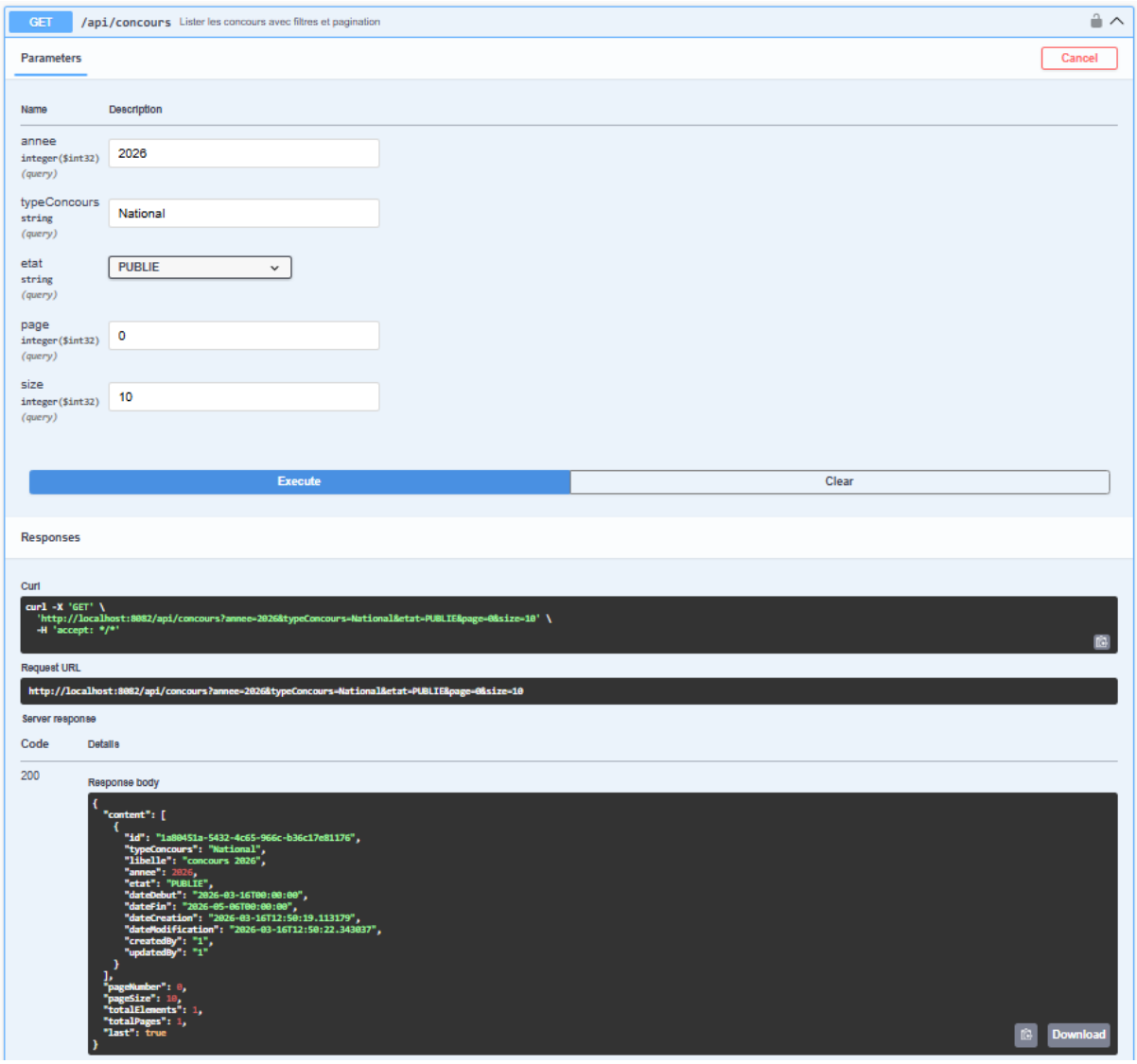

FIGURE 3.11. *Test d'intégration pour afficher la liste filtrée des concours*

3.6 Revue du sprint

3.6.1 Principales interfaces

- **Interface « Gestion globale des concours »**

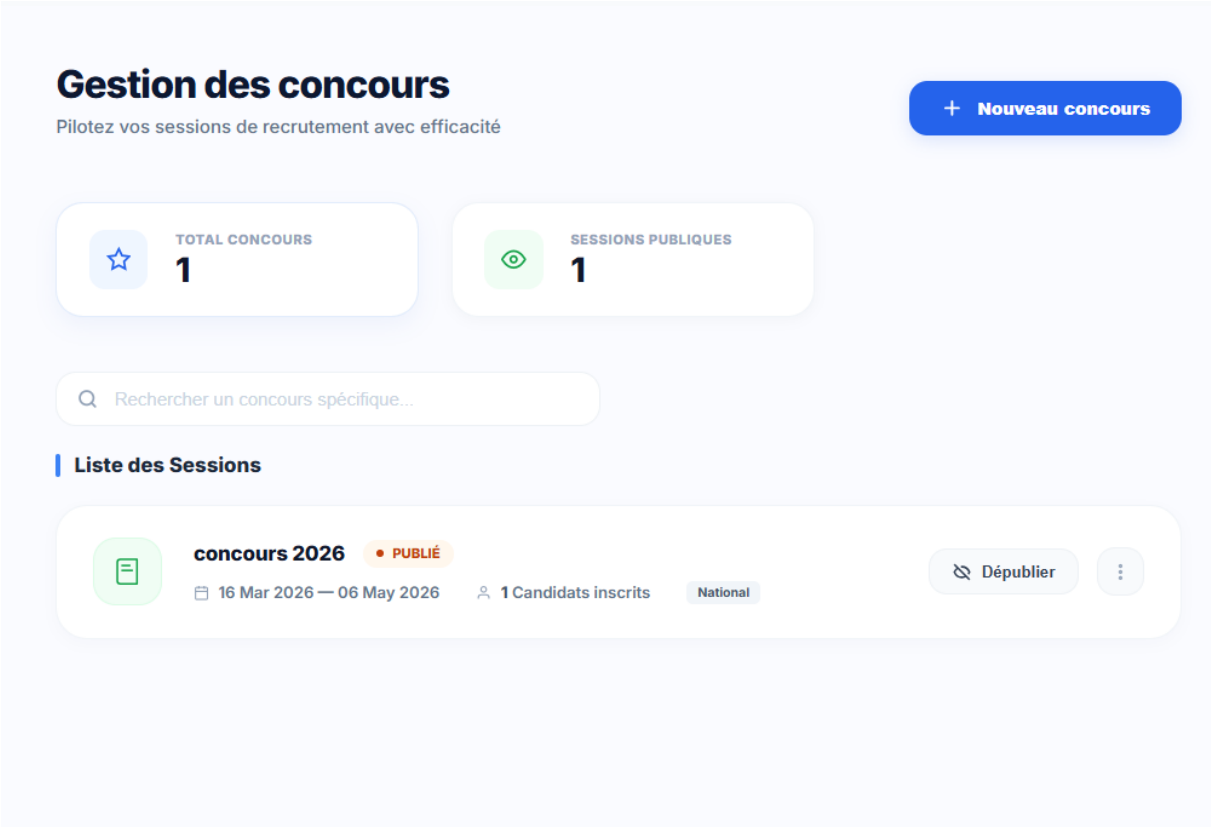

FIGURE 3.12. *Interface de tableau de bord pour la gestion et le suivi des concours*

À partir de cette interface principale, l'administrateur dispose d'un tableau de bord de pilotage centralisé pour gérer efficacement toutes les sessions du concours. S'il souhaite ajouter une session, un bouton d'accès rapide « + Nouveau concours » est mis à sa disposition en haut de la page.

Cette interface a été conçue pour être ergonomique et se divise en plusieurs zones fonctionnelles clés :

— Un aperçu statistique rapide affichant le nombre total de concours enregistrés et le nombre de sessions actuellement rendues publiques.

— Une barre de recherche dynamique, facilitant l'accès direct et ciblé à un concours spécifique au sein de la base de données.

— Une « Liste des Sessions » présentant chaque concours sous forme de carte individuelle détaillée. Celle-ci indique en temps réel l'état d'avancement (ex : Publié), la période de validité (dates de début et de fin), le type d'examen (ex : National) ainsi que le nombre exact de candidats qui y sont inscrits.

Directement depuis cette liste, l'administrateur peut piloter la plateforme et exécuter des actions de gestion rapides sur chaque élément, telles que le fait de « Dépublier » un concours ou encore d'accéder à un menu déroulant d'options supplémentaires (modifier, supprimer) via l'icône dédiée.

• **Interface « Tableau de bord du candidat »**

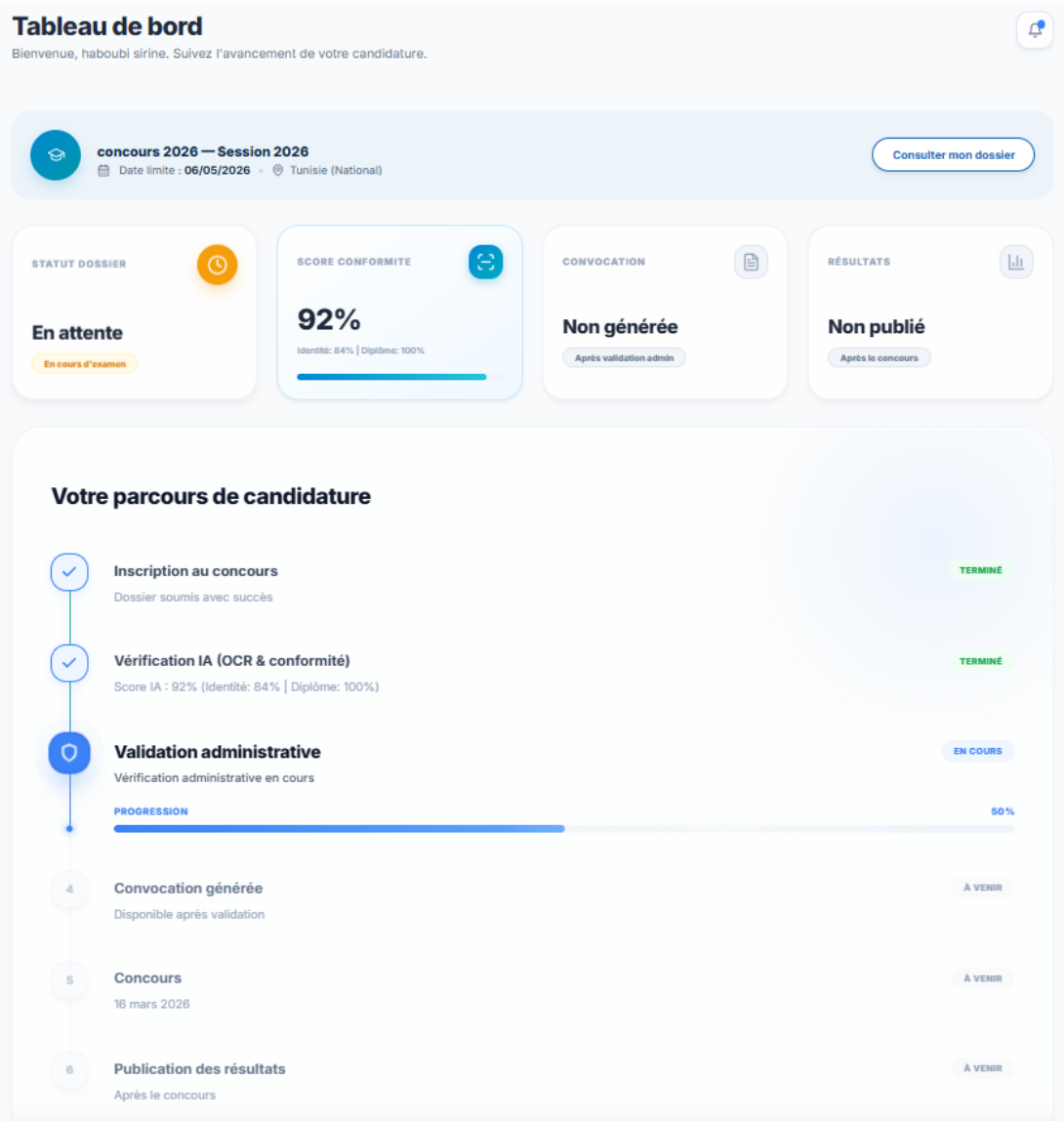

FIGURE 3.13. *Interface de suivi d'avancement pour un candidat connecté*

Cette interface représente l'espace personnel d'un candidat suite à son authentification. Elle a été pensée pour offrir une expérience utilisateur (UX) fluide, en garantissant une vision claire, transparente et en temps réel de l'état d'avancement de sa candidature.

Le tableau de bord s'articule autour de trois éléments fonctionnels majeurs :

— Un récapitulatif du concours auquel le candidat a postulé, accompagné d'un accès rapide pour consulter son dossier complet.

— Un panneau d'indicateurs visuels (KPIs) qui résume le statut actuel du dossier, l'état d'édition de sa convocation, et surtout le **score de conformité** évalué automatiquement par notre module d'Intelligence Artificielle (OCR).

— Une frise chronologique dynamique intitulée « Votre parcours de candidature ». Elle permet au médecin candidat de suivre sa progression étape par étape, depuis l'inscription initiale jusqu'à la publication finale des résultats du concours.

3.7 Mise à jour du product backlog

Le tableau suivant présente le product backlog mis à jour de notre application.

TABLE 3.4. *Product backlog mis à jour*

User stories accomplies dans le sprint 1					33
Ajouter	Intégration Swagger OpenAPI	En tant qu'équipe de développement, nous voulons intégrer Swagger pour tester nos APIs.	Haute	3	Sprint 1
Ajouter	Modifier Dossier	En tant que candidat, je veux modifier mon dossier avant sa validation afin de corriger d'éventuelles erreurs.	Haute	5	
Déplacer	Modifier Profil (US 35)	En tant qu'utilisateur, je veux modifier mon profil afin de mettre à jour mes informations.	Basse	3	De sprint 1 vers sprint 2

3.8 Replanification des sprints

TABLE 3.5. *Replanification des sprints*

N° Sprint	N° User Story	Vélocité
Sprint 1	De 1 à 11	69
Sprint 2	De 12 à 16	23
Sprint 3	De 17 à 18	11
Sprint 4	De 19 à 22	31

3.9 Rétrospective du sprint

3.9.1 Tableau de rétrospective

TABLE 3.6. *Tableau de rétrospective*

Ce qui est bien passé	Ce qui nécessite des améliorations	Actions
L'équipe a réussi dans l'utilisation des microservices, le choix de l'architecture microservice permet une plus grande évolutivité, flexibilité, facilité de maintenance, facilité de déploiement et scalabilité, ce qui peut être très avantageux pour le développement de notre application.	La planification du premier sprint était un peu difficile.	Un bon choix doit être effectué dans un planning poker.
Malgré le retard de planification du sprint, notre équipe a réussi à terminer toutes les tâches du sprint 1.	Un peu de conflit sur le design de quelques interfaces.	Lors du choix des maquettes, toute l'équipe doit être notifiée.
	Difficulté d'intégration du code.	Le partage de code sur le git après chaque terminaison d'un besoin fonctionnel.

3.9.2 ScrumBoard du premier daily

La figure suivante montre l'état du ScrumBoard (tableau Kanban) lors de notre premier *daily*. Toutes les sous-tâches relatives à la User Story **« Déposer un Dossier de Candidature »** se trouvent initialement dans la colonne **« À FAIRE »**. Il en est de même pour les sous-tâches des autres tickets du Sprint 1 (qui ont suivi l'ordre de priorité défini dans notre *Sprint Backlog*).

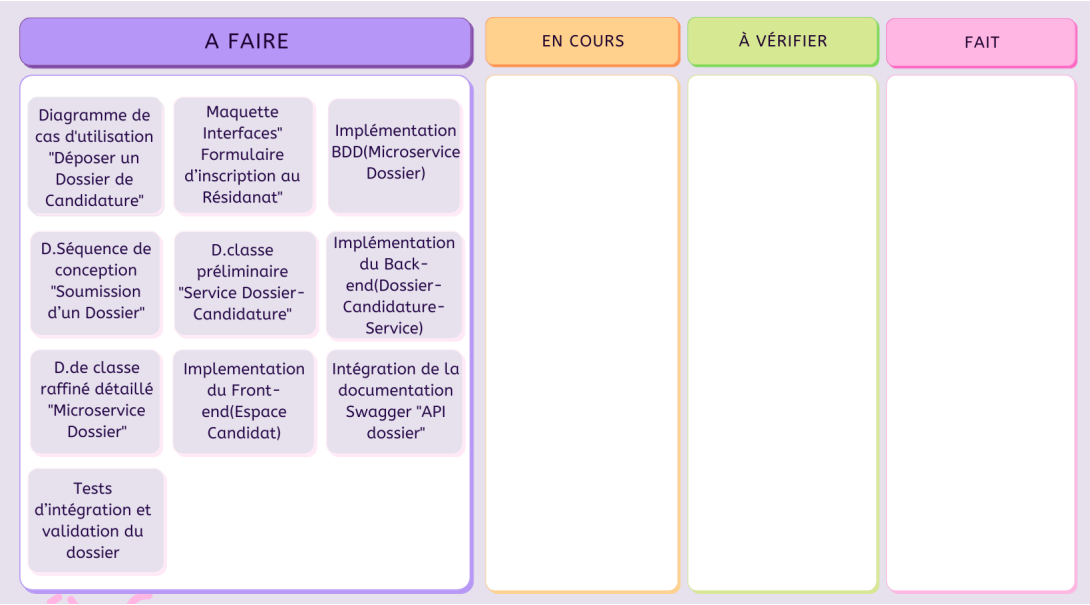

FIGURE 3.14. *ScrumBoard du premier daily*

3.9.3 ScrumBoard du dernier daily

Lors de notre dernier *daily*, toutes les tâches de la User Story **« Déposer un Dossier de Candidature »**, comme l'illustre la figure ci-dessous, ont été déplacées vers la colonne **« FAIT »** du ScrumBoard (il en va de même pour les autres tickets du sprint). Cette évolution démontre l'achèvement complet des cas d'utilisation prévus pour le Sprint 1, et confirme la mise en place réussie du microservice de gestion des dossiers ainsi que son interface candidat.

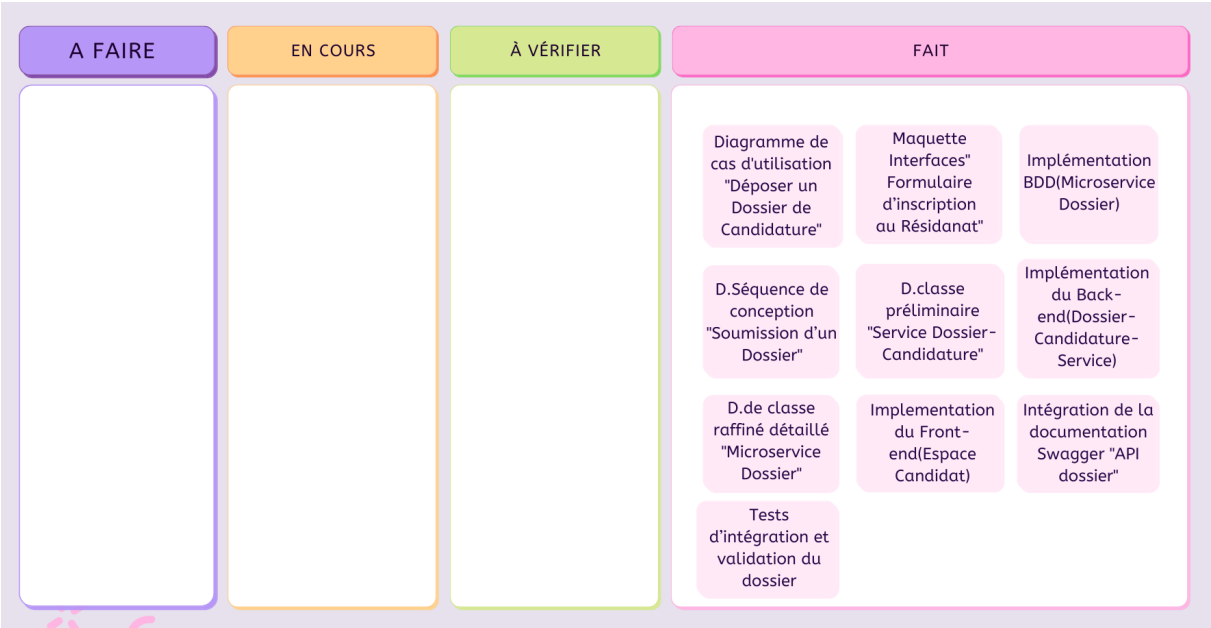

FIGURE 3.15. *ScrumBoard du dernier daily*

3.10 Mesure d'avancement

3.10.1 Burn-down chart

Le Burn-down chart suivant illustre notre progression tout au long du sprint 1 en termes de tâches accomplies et d'effort restant à effectuer.

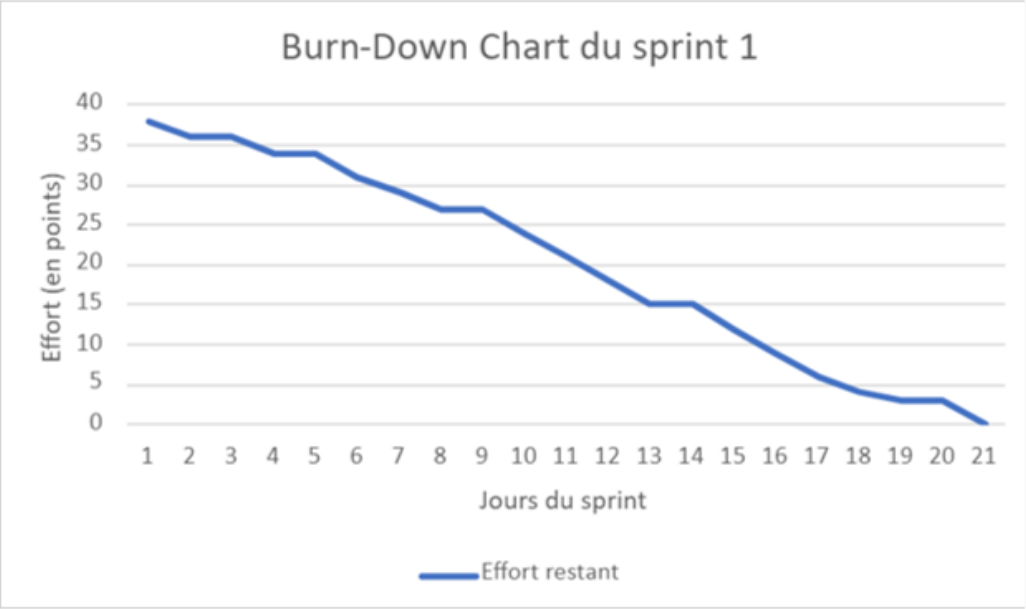

FIGURE 3.16. *Burn-down chart du sprint 1*

3.11 Conclusion

Ce premier sprint a permis de concrétiser les fondations techniques et fonctionnelles de la plateforme « Résidanat TN ». À travers une démarche complète allant de la conception UML à la validation des API via Swagger, nous avons implémenté avec succès les modules d'authentification, de gestion des concours et d'inscription assistée par l'Intelligence Artificielle.

En parallèle, l'application rigoureuse des pratiques Scrum (mise à jour du backlog, ScrumBoard, Burn-down chart et rétrospective) a assuré un suivi transparent de notre progression. Cette bonne maîtrise de notre architecture distribuée en micro-services nous permet d'aborder sereinement la suite du développement.

Le chapitre suivant détaillera le deuxième sprint, qui sera focalisé sur le traitement des dossiers soumis, la gestion des alertes générées par l'IA et la validation des candidatures par l'administration.